Oregon State University
Ecampus

CS361: Assignment 6: Sprint 2 Plan (for Microservice A)

Charles Gilbert & Mohana Vista & Nathan Zak

Sprint Goal

Sprint Goal: Fully implement the Data Validation Service, Data Search/Filter Service, and CSV to JSON Converter.

Plan for Microservice 1: Data Validation Service

1. Which teammates will specify, design, and implement this microservice?

• Charles Gilbert and Mohana Vista are responsible for the specification and design. • Charles Gilbert, Mohana Vista and Nathan Zak will implement this microservice in their main programs.

2. Provide a **link** to the repository.

https://github.com/m-vista/DU-DataValidation

3. Write at least three user stories for this Sprint.

First user story

(Front of index card) Required Field Validation *As a developer, I want to check that required fields contain values so that my application can reject incomplete records.*
(Back of index card) Acceptance criteria Functional requirements • Given a required field is missing or empty, when the program sends the record and the rules to the data validation service, then it will identify the missing field and send a “missing field” error message. • Given all required fields contain values, when the record is validated, then no required field error is returned.

Quality attributes & Non-functional requirements

- Reliability: The service passes all 10 prepared required field test cases (null, empty and present)

Second user story

(Front of index card)

Data Type Validation

As a developer, I want to check whether input fields contain the expected data type so that my application can avoid processing the wrong data types.

(Back of index card)

Acceptance criteria

Functional requirements

- Given a record containing values that match their specified data types, when the microservice validates the record, then it returns a successful validation result.
- Given a record containing a value that does not match its specified data type, when the microservice validates the record, then it returns an unsuccessful validation result identifying the invalid field.

Quality attributes & Non-functional requirements

- Usability: Each invalid-type test response includes a readable message naming the affected field.

Third user story

(Front of index card)

Numerical Range Validation

As a developer, I want to set minimum and maximum numeric values so that my application can reject numbers outside its allowed range.

(Back of index card)

Acceptance criteria

Functional requirements

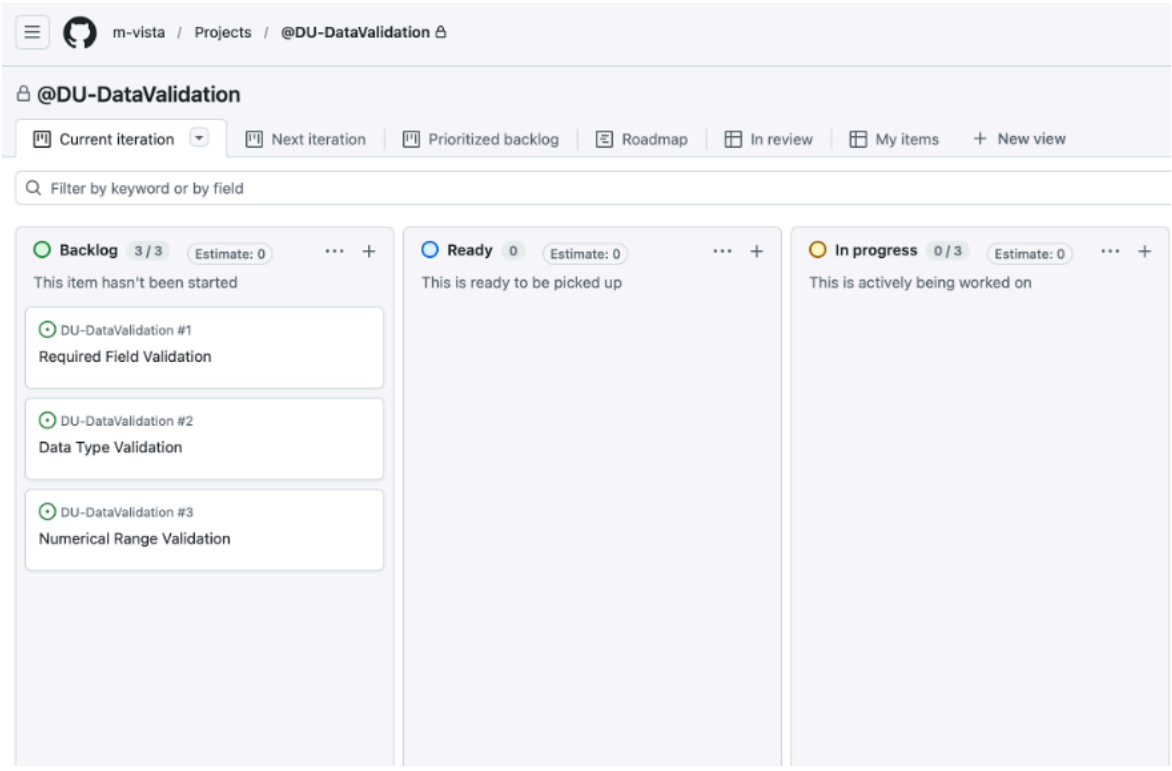
- Given a numeric field has a set minimum value and contains a number less than that value, when the microservice validates it, then the service invalidates it with an error stating the minimum.
- Given a numeric field has a set maximum value and contains a number greater than that value, when the microservice validates it, then the service invalidates it with an error stating the maximum.
- Given a numeric field has a set minimum and maximum, and the value is within

the range, when it is validated, then no range error is returned.

Quality attributes & Non-functional requirements

- Reliability: The service passes 10 boundary tests (below, above, at, and within the set limits) and returns the expected results.

4. Take a **screenshot** that shows the user stories are in a Sprint Backlog in GitHub Projects.



5. What kind of **communication pipe** will the microservice use? (e.g., text files, REST API)

ZeroMQ using TCP sockets and JSON messages.

6. How will other programs be able to **request data** from the microservice and what request parameters will be required?

Request parameters:

- data: JSON object
- rules: JSON object
- required: Boolean
- type: String
- min: Number
- max: Number

Detailed description:

Other programs will connect to the microservice using a ZeroMQ REQ socket. The requesting program will send a JSON object containing the data to validate and the validation rules. The microservice will check the required fields, data types, and numerical ranges.

Example call (Node.js): How express backend can request validation

```
const zmq = require("zeromq");

async function validateData() {
  const socket = new zmq.Request();
  socket.connect("tcp://127.0.0.1:5555");

  const request = {
        data: {
        breastMilk: 3.5,
        formula: 1.0
        },
        rules: {
        breastMilk: {
        required: true,
        type: "number",
        min: 0
        },
        formula: {
        required: true,
        type: "number",
        min: 0
        }
        }
  };

  try {
        await socket.send(JSON.stringify(request));
        const [reply] = await socket.receive();
        return JSON.parse(reply.toString());
  } finally {
        socket.close();
  }
}
```

7. How will other programs be able to **receive data** from the microservice and what data will the microservice provide?

Data type: JSON object, transmitted through ZeroMQ.
Detailed description:

After receiving the request, the microservice validates the submitted data and sends a JSON response through it ZeroMQ REP socket. The response contains three fields (valid (Boolean), data (original data is validated, null if failed), errors (list of error messages if validation fails))

Example call (Node.js):

```
const result = await validateData();

if (result.valid) {
  console.log("Validation successful");
  console.log(result.data);
} else {
  console.log("Validation failed");
  console.log(result.errors);
}
```

Example successful response:

```
{
  "valid": true,
  "data": {
      "breastMilk": 3.5,
      "formula": 1.0
  },
  "errors": []
}
```

Example unsuccessful response:

```
{
  "valid": false,
  "data": null,
  "errors": [
      "breastMilk must be at least 0"
  ]
}
```

Plan for Microservice 2: Data Search/Filter Service

1. Which teammates will specify, design, and implement this microservice?

   Nathan Zak and Mohana Vista will design this microservice; both they and Charles Gilbert will implement it.

2. Provide a **link** to the repository.

   https://github.com/m-vista/DU-DataFilter/projects

3. Write at least three user stories for this Sprint.

   First user story

   (Front of index card)

Filter Data by String

As a user, I want to be able to filter my data with a string so that I can quickly see the relevant entries.

(Back of index card)
Acceptance criteria

Functional requirements

- Given I'm filtering data, when I filter for a specific string then I will only see the entries that contain that exact string

Quality attributes & Non-functional requirements

- Performance:
  - The service must filter every correlating entry from a dataset of 1000 entries in less than 2 seconds
  - Algorithm runs at O(n) or better

Second user story

(Front of index card)

Handle Invalid Filter Request

As a user, I want to be informed when no entries match my filter request so that I can adjust my request and try again, or work with that knowledge as a result of its own.

(Back of index card)
Acceptance criteria

Functional requirements

- Given I'm filtering my data, when I request a filter that doesn't match an entry in my dataset then I will receive an error message from the service that no results were found
- Given I'm filtering data, when I request an empty filter then I will receive an error message from the service telling me to enter criteria and try again

Quality attributes & Non-functional requirements

- Performance:
  - The service must filter every correlating entry from a dataset of 1000 entries in less than 2 seconds
  - Algorithm runs at O(n) or better

Third user story

(Front of index card)
Filter Data by Integer

As a user, I want to be able to filter my data by an integer or range of integers so that I can quickly see all the relevant entries.

(Back of index card)
Acceptance criteria

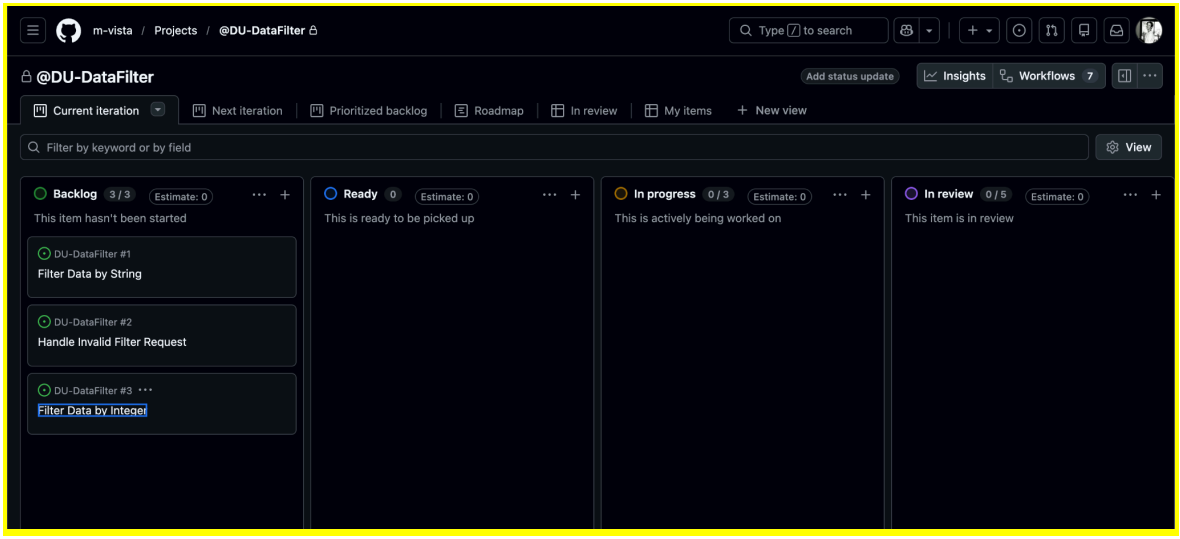
Functional requirements

- Given I'm filtering data, when I filter by a specific integer then I will see every entry with that exact integer
- Given I'm filtering data, when I filter by a specific range of integers then I will see every entry within the range of those integers

Quality attributes & Non-functional requirements

- Performance:
  - The service must filter every correlating entry from a dataset of 1000 entries in less than 2 seconds
  - Algorithm runs at O(n) or better

4. Take a screenshot that shows the user stories are in a Sprint Backlog in GitHub Projects.



5. What kind of **communication pipe** will the microservice use? (e.g., text files, REST API)

ZeroMQ sending JSON strings through REQ/REP sockets

6. How will other programs be able to **request data** from the microservice and what request parameters will be required?

JSON strings
Applications will request data through ZeroMQ REQ sockets using JSON string as parameters
create ZeroMQ request socket connect it to the filter service (port: xxxx) build a request message with: (key) is (value) (key) is (value) ... convert request message into JSON string send the JSON string through the socket, wait for a reply turn the reply from JSON back into data if the reply is “ok”, use matching data else, show error message

7. How will other programs be able to **receive data** from the microservice and what data will the microservice provide?

JSON strings
A status field in a JSON string will either send “ok” and the correlating data or “error” and an explanatory message
receive a JSON response from a ZeroMQ socket parse the message if status is “ok”: read and display all relevant data else, show error message

Plan for Microservice 3: CSV to JSON Converter

1. Which teammates will specify, design, and implement this microservice?

Charles Gilbert and Nathan Zak will specify and design the CSV-to-JSON microservice and collaborate on its implementation. Charles Gilbert, Mohana Vista and Nathan Zak will implement this microservice in their main programs.

2. Provide a **link** to the repository.

https://github.com/m-vista/DU-Csv2Json

3. Write at least three user stories for this Sprint. Provide your user stories and their functional and non-functional acceptance criteria (and associated quality attributes).

First user story

(Front of index card)
Convert CSV Data to JSON

As a developer, I want to convert CSV data into JSON so that my application can process and exchange data in a standardized format.

(Back of index card)
Acceptance criteria

Functional requirements

- Given the CSV-to-JSON microservice is running and receives valid CSV data containing a header and two records, when a conversion is requested, then the service returns a successful JSON response containing both records with the correct field names and values.

Quality attributes & Non-functional requirements

- Performance: The service must convert a valid CSV dataset containing 100 records and five columns in under two seconds in each of 10 consecutive local tests.

Second user story

(Front of index card)
Support Different CSV Column Names

As a developer, I want the microservice to support different CSV column names so that I can convert data from different applications without modifying the service.

(Back of index card)
Acceptance criteria

Functional requirements

- Given valid CSV data with column names that differ from previous requests, when the service converts the data, then the returned JSON records preserve the column names and corresponding values from the submitted CSV.

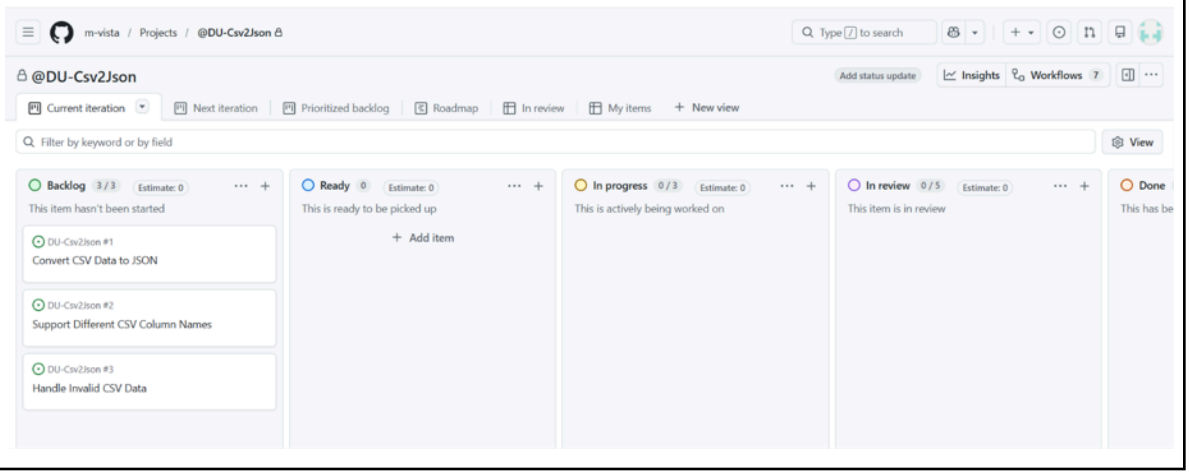
Quality attributes & Non-functional requirements

- Maintainability: The microservice must successfully convert CSV datasets with three different sets of column headers without requiring any changes to the conversion code.

Third user story

(Front of index card) **Handle Invalid CSV Data** *As a developer, I want to receive a clear error message when CSV data is invalid so that my application can identify problems and avoid processing incorrect data.*
(Back of index card) Acceptance criteria Functional requirements ● Given CSV data containing rows with a different number of fields than the header, when a conversion is requested, then the service returns an error status and a message explaining the problem. ● Given an empty CSV input, when a conversion is requested, then the service returns an error status indicating that no CSV data was provided. Quality attributes & Non-functional requirements ● Reliability: The service must return the expected error response for all 10 prepared invalid-input test cases without crashing.

4. Take a **screenshot** that shows the user stories are in a Sprint Backlog in GitHub Projects.



5. What kind of **communication pipe** will the microservice use? (e.g., text files, REST API)

ZeroMQ using the REQ/REP (Request/Reply) communication pattern. Messages will be exchanged as JSON-formatted strings.

6. How will other programs be able to **request data** from the microservice and what request

parameters will be required?

Request Parameters:

- `action`: String specifying the requested operation (convert).
- `csv_data`: String containing the CSV header and records to convert.

Detailed Description:

The main program creates a ZeroMQ REQ socket and connects to the CSV-to-JSON microservice on an agreed local port. It sends a JSON request containing the CSV data. The microservice receives the request and performs the conversion.

Example Pseudocode:

```
Create ZeroMQ REQ socket
Connect to CSV-to-JSON service (agreed port)

Create request:
        action = "convert"
        csv_data = CSV header and records

Convert request into JSON string
Send request to microservice
Wait for response
```

7. How will other programs be able to **receive data** from the microservice and what data will the microservice provide?

Data Type:

JSON-formatted string containing the conversion status and either the converted records or an error message

Detailed Description:

After receiving the request, the microservice parses the CSV data and converts each row into a JSON object using the column headers as field names.
If conversion succeeds, the service returns an ok status and a list of converted records.
If conversion fails, the service returns an error status and an explanatory message.

Example Successful Response:

```
{
```

```
  "status": "ok",
  "data": [
    {"patient_id": "P001", "test": "Glucose", "result": "95"},
    {"patient_id": "P002", "test": "Glucose", "result": "110"}
  ]
}
```

Example Error Response:

```
{
  "status": "error",
  "message": "CSV row has an incorrect number of fields."
}
```

Example Pseudocode:

```
Receive JSON response from ZeroMQ socket
Parse response

If status is "ok":
        Read and use converted JSON records
Else:
        Display the error message
```

Teammate Contributions

1. Mohana proposed a plan for the Data Validation Service, and Charles reviewed and finalized it.
2. Nathan proposed a plan for the Data Search/Filter Service, and Mohana reviewed and finalized it.
3. Charles proposed a plan for the CSV-to-JSON converter, and Nathan reviewed and finalized it.
4. Mohana created the GitHub repos and projects and added user stories to the backlog.
5. Mohana, Charles, and Nathan peer reviewed each others' drafts and made collaborative changes where needed
6. Mohana created a group doc for the project and filled in the Sprint Goal and Data Validation Service section.
7. Charles filled in the CSV to JSON Service section.
8. Nathan filled in the Data Search/Filter Service section.
9. Nathan updated the teammate contributions & submitted the document on 11/8/26.